

Übersetzer-App

Entwicklung einer Offline-Übersetzungs-App mit Capacitor und Google ML Kit

Fallstudie Hybride App-Entwicklung · WWI23-6 · 19.08.2026

Menko Hornstein · Lisa Fritsche · Jan Goliasch

DE

EN

FR

ES

IT

Was Sie in den nächsten 12 Minuten sehen

1

Die App

Funktionen und Anforderungen

2

Technik & Architektur

Stack, Schichten, Bridge

3

Die nativen Plugins

Funktionsweise der Capacitor Bridge

4

Im Code

Ausgewählte stellen des Codes

5

Probleme & Grenzen

Technische Herausforderungen und Einschränkungen

6

Live-Demo

Übersetzen und Modellverwaltung

Aufgabenverteilung im Projekt

A

Menko Hornstein

Native-Integration & Build

- Plugins ausgewählt und eingebunden
- Capacitor- und Gradle-Konfiguration
- Statusleiste und Spracherkennung
- Gerätetests und APK-Build

```
main.ts  
MainActivity.java  
app/build.gradle
```

B

Jan Goliasch

Übersetzer-Oberfläche

- Eingabe, Ergebnis, Ladezustände
- Zwischenablage mit Toast
- Text-to-Speech inkl. Stimmenprüfung
- Tauschen von Sprachen und Texten

```
HomePage.vue
```

C

Lisa Fritsche

Modellverwaltung & Struktur

- Ansicht /models
- Zentrale Sprachliste für beide Views
- Router und Navigation
- Dokumentation und Foliensatz

```
ModelsPage.vue  
router/index.ts  
languages.ts
```

Fünf Sprachen, komplett offline

13

von 13 Grundanforderungen
umgesetzt

Deutsch · Englisch · Französisch
Spanisch · Italienisch

A

Übersetzen

Fünf Sprachen in beide
Richtungen

B

Erkennen

Quellsprache automatisch
bestimmen

C

Vorlesen

Zieltext über die TTS-Engine

D

Kopieren

Zieltext in die Zwischenablage

E

Tauschen

Quell- und Zielsprache inklusive
Texten tauschen

F

Verwalten

Sprachmodelle laden und löschen

Technologie-Stack

Oberfläche	Vue 3 + Ionic 8	Composition API, Komponenten
Build	Vite 5 + TypeScript	Dev-Server, Typsicherheit
Native Brücke	Capacitor 8	Web-App wird zur Android-App
Übersetzung	mlkit/translation	On-Device, ein Modell je Sprache
Spracherkennung	mlkit/language-id	Liefert „und“ bei Unsicherheit
Geräte-APIs	Clipboard · TTS	Fertige Plugins aus npm
Statusleiste	@capacitor/status-bar	Platz für Uhrzeit und Symbole

?

Warum Capacitor?

- Die Oberfläche läuft in einer WebView — wir konnten unser Web-Wissen direkt einsetzen.
- Gemeinsamer Codebestand mit Erweiterungsmöglichkeit für iOS
- Native Funktionen erfordern geeignete Capacitor-Plugins und bildeten einen Schwerpunkt der Implementierung

Drei Schichten, eine Brücke



Alle fünf Plugins werden über npm eingebunden. Im Verzeichnis android/ ist daher kein eigener Java-Code erforderlich; die MainActivity bleibt unverändert

Ein Capacitor-Plugin besteht aus drei Teilen

1 Native Seite

node_modules/.../android/

```
@CapacitorPlugin(  
  name = "Translation")  
public class TranslationPlugin  
  extends Plugin {  
  @PluginMethod  
  public void translate(  
    PluginCall call) { ... }
```

@PluginMethod macht jede Methode von JavaScript aus aufrufbar. PluginCall dient sowohl der Übergabe der Eingabedaten als auch zur Rückgabe des Ergebnisses: resolve() oder reject().

2 Registrierung

automatisch beim Sync

```
npx cap sync android  
  
# trägt die Plugins ein in  
#   capacitor.settings.gradle  
#   capacitor.build.gradle
```

Plugins aus npm findet Capacitor selbst. Nur ein eigenes Plugin im App-Projekt müsste man in MainActivity per registerPlugin() anmelden.

3 TypeScript-Seite

src/views/HomePage.vue

```
import { Translation }  
  from "@capacitor-mlkit/...";  
  
Translation.translate({  
  text, sourceLanguage,  
  targetLanguage });
```

Intern ruft das Plugin registerPlugin("Translation") auf. Der Plugin-Name muss exakt mit der Java-Annotation übereinstimmen, damit die Zuordnung zwischen TypeScript- und nativer Ebene funktioniert.

Sprachmodelle für die Offline-Übersetzung

~ 30 MB

pro Sprache auf dem Gerät

1 Modell

je Sprache, nicht je Sprachpaar —
Englisch dient intern als Brücke

Wie wir die Modelle verwalten

1

Drei Plugin-Methoden

Die Modellverwaltung basiert auf den drei Methoden `getDownloadedModels`, `downloadModel` und `deleteDownloadedModel`

2

Eigene Ansicht

Route `/models` zeigt für jede der fünf Sprachen „Installiert“ oder „Nicht installiert“, mit Laden- und Löschen-Button.

3

Status kommt vom Gerät

Nach jedem Download oder Löschen wird der aktuelle Modellstatus erneut vom Gerät abgefragt

4

Nur die aktive Zeile sperrt

Während eines Downloads wird nur der betroffene Eintrag deaktiviert; die übrigen Modelle können weiterhin verwaltet werden.

Was beim Klick auf „Übersetzen“ passiert

1

Eingabe prüfen

Text getrimmt,
Meldungen geleert,
Spinner an

2

Sprache erkennen

Nur bei
„Automatisch“:
identifyLanguage

3

und?

Plugin liefert "und" ;
Vue zeigt Hinweis

4

Übersetzen

ML Kit übersetzt lokal
auf dem Gerät

5

Anzeigen

Ergebniskarte kommt,
Spinner aus

!

Jeder Bridge-Aufruf ist asynchron

Jeder Bridge-Aufruf erfolgt asynchron und wird daher mit try / catch / finally behandelt. Der Ladezustand wird im finally-Block zuverlässig zurückgesetzt. Die Ladeanzeige ist direkt im jeweiligen Button integriert, sodass die aktuell ausgeführte Aktion eindeutig erkennbar bleibt.

Spracherkennung: der Fall „und“

1 Erkennen

HomePage.vue · translateText()

```
const result =  
  await LanguageIdentification  
    .identifyLanguage({  
      text: cleanedText,  
    });
```

Läuft nur, wenn die Ausgangssprache auf „Automatisch erkennen“ steht. Der Aufruf geht über die Capacitor-Bridge an ML Kit.

2 Der Fall und

dokumentierter Rückgabewert

```
const lang = result.language;  
  
if (lang === "und") {  
  errorMessage.value =  
    "Sprache nicht erkannt."  
  return;  
}
```

ML Kit liefert „und“ für undetermined — kein Fehler, sondern ein normaler Rückgabewert. Deshalb steht die Prüfung hier und nicht im catch-Block.

3 Nur unsere fünf

src/data/languages.ts

```
const detected =  
  languages.find(  
    (l) => l.code === lang,  
  );  
  
if (!detected) { ... }
```

ML Kit erkennt über 100 Sprachen, übersetzen wollen wir fünf. Wird etwas anderes erkannt, sagen wir das offen, statt es zu versuchen.

Sprachausgabe: Zustand über Events

1 Das Problem

Plugin-Dokumentation

```
// speak() löst auf, sobald  
// die Äußerung in der  
// Warteschlange liegt –  
// nicht wenn sie endet
```

Ein `await` auf `speak()` sagt nichts über das Ende der Ausgabe. Ohne Gegenmaßnahme verschwindet der Stopp-Button sofort wieder.

2 Die Lösung

`onMounted()` in `HomePage.vue`

```
addListener("start", () => {  
  isSpeaking.value = true;  
});  
  
addListener("end", () => {  
  isSpeaking.value = false;  
});
```

Start und Ende der Ausgabe steuern den Zustand. Der Stopp-Button ist genau so lange sichtbar, wie tatsächlich gesprochen wird.

3 Der Aufruf

`speakTranslation()`

```
await SpeechSynthesis.speak({  
  text: translatedText.value,  
  language: speechTag,  
  queueStrategy:  
    QueueStrategy.Flush,  
});
```

`language` will BCP-47 mit Region (de-DE), ML Kit nur de — `languages.ts` führt beide Werte. `Flush` unterbricht eine laufende Ausgabe.

Technische Herausforderungen und Lösungsansätze

1 Überlagerung durch Statusleiste

Die WebView überlagerte zunächst die Android-Statusleiste. Behoben wurde dies durch `StatusBar.setOverlaysWebView({ overlay: false })` in `main.ts`.

2 Fehlende TTS-Ausgabe

Die TTS-Engine erwartet BCP-47-Tags wie `de-DE` anstelle des ML-Kit-Codes `de`. Daher werden beide Sprachcodes getrennt verwaltet und die Unterstützung vor der Ausgabe geprüft.

3 Fehler beim Sprachtausch mit automatischer Erkennung

`auto` kann nicht als Zielsprache verwendet werden. Beim Tauschen wird daher die zuletzt erkannte Sprache übernommen; ist keine vorhanden, wird ein Hinweis ausgegeben

Aktuelle Einschränkungen der App

Android getestet

Die Anwendung wurde ausschließlich unter Android getestet. Eine iOS-Version wäre mit den verwendeten Plugins grundsätzlich möglich, wurde im Projekt jedoch nicht umgesetzt.

- 👤 iOS ergänzen: npx cap add ios
- 👤 Kein nativer Code nötig – Plugins bringen Swift mit
- 👤 Voraussetzung: Mac, Xcode, iOS 15.5

Internetverbindung für initialen Modelldownload

Die Übersetzung funktioniert offline. Für den erstmaligen Download eines Sprachmodells ist jedoch eine Internetverbindung erforderlich.

- 👤 Modelle beim ersten Start im WLAN vorladen
- 👤 Netzstatus prüfen via @capacitor/network
- 👤 Fortschrittsbalken statt unbestimmtem Spinner

Fünf Sprachen

ML Kit kann über 50 übersetzen und über 100 erkennen. Wir prüfen erkannte Sprachen gegen unsere Liste.

- 👤 Liste aus dem Language-Enum generieren
- 👤 Anzeigenamen über Intl.DisplayNames
- 👤 Suchfeld in der Sprachauswahl

Abhängigkeit von installierten TTS-Stimmen

Die Sprachausgabe ist von den auf dem Gerät installierten TTS-Stimmen abhängig. Fehlende Stimmen müssen über die Android-Einstellungen nachinstalliert werden.

- 👤 Stimmen schon in der Sprachauswahl prüfen
- 👤 Fehlende dort markieren statt erst beim Vorlesen
- 👤 Verweis in die Android-Sprachausgabe-Einstellungen

Live-Demo

1

Modelle prüfen

Modellverwaltung öffnen und den Installationsstatus der Sprachmodelle prüfen

2

Modell laden

Französisches Sprachmodell herunterladen und die Statusänderung auf „Installiert“ demonstrieren

3

Übersetzen

Deutschen Text ins Französische übersetzen und anschließend die Offline-Funktion im Flugmodus demonstrieren

4

Erkennen & Tauschen

Quellsprache auf „Automatisch“, englischen Satz eingeben. Danach tauschen.

5

Vorlesen & Kopieren

Zieltext über die TTS-Funktion ausgeben und anschließend in die Zwischenablage kopieren.

FAZIT

We made it!

Vielen Dank — Fragen?